



(4) W_0 的大小表示螺线的伸展率。 $W_0>1$ 时, 随着 k 的增加螺线

内缩; $W_0<1$ 时, 则随 k 的增加螺线外伸; $W_0=1$ 时, 表示是半

径为 A_0 的一段圆弧。若又有 $A_0=1$, 则这段圆弧是单位圆的

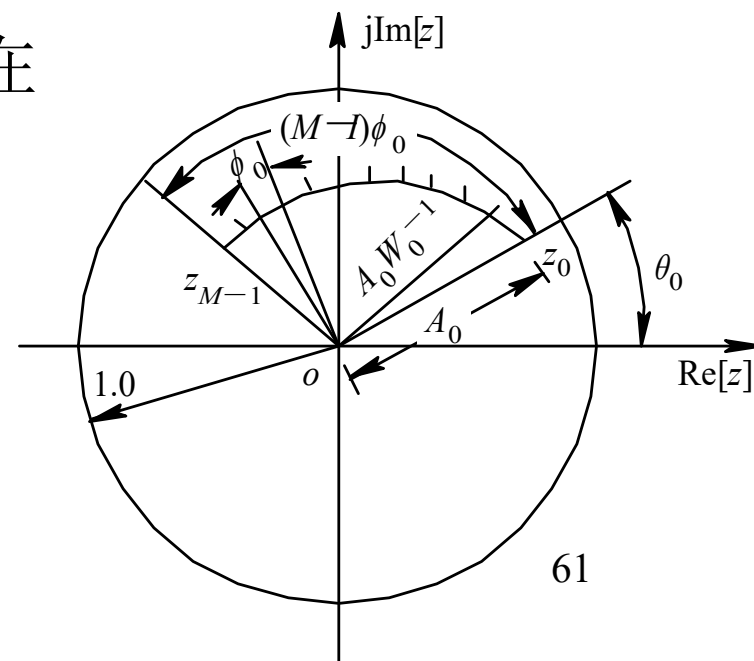
一部分。

当 $M=N$, $A=A_0e^{j\theta_0}=1$, $W=W_0 \cdot e^{-j\varphi_0} = -j\frac{2\pi}{N}$ ($W_0=1$, $\varphi_0=2\pi/N$) 这

一特殊情况时, 各 z_k 就均匀等间隔地分布在

单位圆上, 这就是求序列的DFT。

将式 (4-22) z_k 的代入Z变换表达式, 可得





$$\begin{aligned} X(z_k) &= \sum_{n=0}^{N-1} x(n) z_k^{-n} \\ &= \sum_{n=0}^{N-1} x(n) A^{-n} W^{nk} \quad 0 \leq k \leq M-1 \end{aligned} \quad (4-25)$$

直接计算这一公式，与直接计算DFT相似，总共算出 M 个采样点，需要 NM 次复数乘法与 $(N-1)M$ 次复数加法。当 N ， M 很大时，这个量很大，这就限制了运算速度。但是，下面我们将看到，通过一定的变换，以上运算可以转换为卷积形式，从而可以采用FFT算法，这样就可以大大提高运算速度。



nk 可以用以下表达式来替换

$$nk = \frac{1}{2}[n^2 + k^2 - (k-n)^2] \quad (4-26)$$

则 (4-25) 式可改写为

$$\begin{aligned} X(z_k) &= \sum_{n=0}^{N-1} x(n) A^{-n} W^{\frac{n^2}{2}} W^{-\frac{(k-n)^2}{2}} W^{\frac{k^2}{2}} \\ &= W^{\frac{k^2}{2}} \sum_{n=0}^{N-1} \left[x(n) A^{-n} W^{\frac{n^2}{2}} \right] W^{-\frac{(k-n)^2}{2}} \end{aligned} \quad (4-27)$$

如果定义:

$$g(n) = x(n) A^{-n} W^{\frac{n^2}{2}} \quad (4-28)$$

$$h(n) = W^{-\frac{n^2}{2}} \quad (4-29)$$



则它们的卷积为

$$\begin{aligned} g(k) * h(k) &= \sum_{n=0}^{N-1} g(n)h(k-n) \\ &= \sum_{n=0}^{N-1} [x(n)A^{-n}W^{\frac{n^2}{2}}]W^{-\frac{(k-n)^2}{2}} \\ k &= 0, 1, \dots, M-1 \end{aligned} \quad (4-30)$$

式（4-30）正好是式（4-27）的一部分，因此式（4-27）又可以用卷积的形式表示为

$$X(z_k) = W^{\frac{k^2}{2}} \cdot [g(k) * h(k)] \quad k=0, 1, \dots, M-1$$



由上式可看出，如果我们对信号按式（4-28）先进行一次加权处理，加权系数为 $A^{-n}W^{\frac{n^2}{2}}$ ；然后，通过一个单位脉冲响应为 $h(n)$ 的线性系统即求 $g(n)$ 与 $h(n)$ 的线性卷积；最后，对该系统的前 M 点输出再做一次加权，这样就得到了全部 M 点螺旋线采样值 $X(z_n)(n=0, 1, \dots, M-1)$ 。这个过程可以用图4.21 表示。

从图中我们看到，运算的主要部分是由线性系统来完成的。由于系统的单位脉冲响应 $h(n) = W^{\frac{n^2}{2}}$ 可以想象为频率随时间(n)呈线性增长的复指数序列。在雷达系统中,这种信号称为线性调频信号 (Chirp Signal)，因此，这里的变换称为线性调频Z变换。

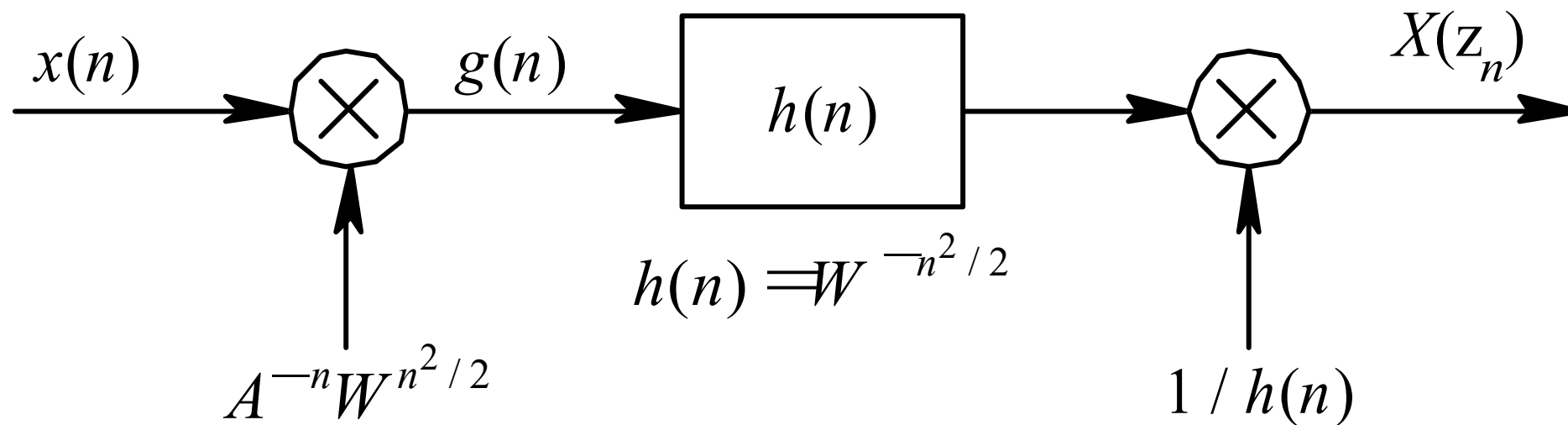


图 4.21 Chirp-Z变换的线性系统表示



4.6.2 Chirp-Z变换 (CZT) 的实现步骤

由式(4-30)可看出, 线性系统 $h(n)$ 是非因果的, 当 n 的取值为0到 $N-1$, k 的取值为 $0, 1, \dots, M-1$ 时, $h(n)$ 是在 $n=-(N-1)$ 到 $n=M-1$ 取值。也就是说, $h(n)$ 是一个有限长序列, 点数为 $N+M-1$ 。

输入信号 $g(n)$ 也是有限长序列, 点数为 N 。 $g(n)*h(n)$ 的点数为 $2N+M-2$, 因而用圆周卷积代替线性卷积且不产生混叠失真条件是圆周卷积的点应大于或等于 $2N+M-2$ 。

由于我们只需要前 M 个值 $X(z_k)$ ($k=0, 1, \dots, M-1$), 对以后的其他值是否有混叠失真并不感兴趣, 这样可将圆周卷积的点数缩减到最小为 $N+M-1$ 。



当然，为了进行基-2 FFT运算，圆周卷积的点数应取为 $L \geq N+M-1$ ，同时又满足 $L=2^m$ 的最小 L 。这样可将 $h(n)$ 先补零值点，补到点数等于 L ，也就是从 $n=M$ 开始补 $L-(N+M-1)$ 个零值点，补到 $n=L-N$ 处，或补 $L-(N+M-1)$ 个任意序列值，然后将此序列以 L 为周期而进行周期延拓，再取主值序列，从而得到进行圆周卷积的一个序列。进行圆周卷积的另一个序列只需要将 $g(n)$ 补上零值点，使之成为 L 点序列即可。

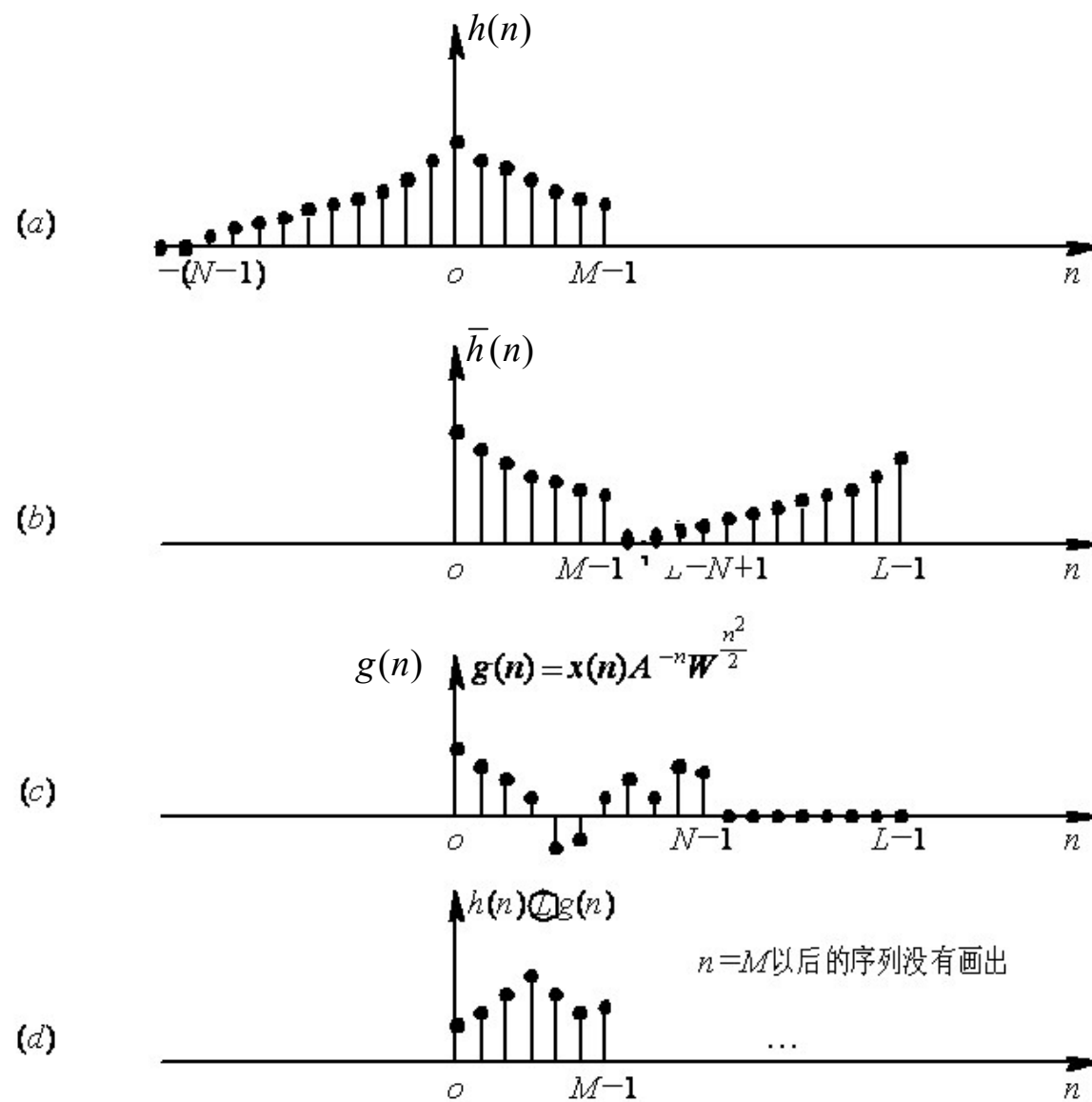


图4.22 Chirp-Z变换的圆周卷积图



这样，我们可以列出**CZT**运算的实现步骤：

(1) 求 $h(n)$ 的主值序列

$$\bar{h}(n) = \begin{cases} W^{-\frac{n^2}{2}}, & 0 \leq n \leq M-1 \\ 0, & M \leq n \leq L-N \\ W^{\frac{(n-L)^2}{2}}, & L-N+1 \leq n \leq L-1 \end{cases}$$

(2) $H(k) = \text{FFT}[\bar{h}(n)]$, L 点

(3) 对 $x(n)$ 加权并补零



$$g(n) = \begin{cases} x(n) A^{-n} W^{\frac{n^2}{2}}, & 0 \leq n \leq M-1 \\ 0, & N \leq n \leq L-1 \end{cases}$$

(4) $G(k) = \text{FFT}[g(n)]$, L 点

(5) $Y(k) = G(k)H(k)$, L 点

(6) $y(n) = \text{IFFT}[Y(k)]$, L 点



$$(7) X(z_k) = W^{\frac{k^2}{2}} y(k), 0 \leq k \leq M-1$$

4.6.3 运算量的估计

其中，（1）、（2）两步是事先准备好的，不必实时计算，因此以上计算包括（3）、（7）两个加权（ $N+M$ 次乘），一个 $Y(k)$ 形成（ L 次乘）以及一个FFT及一个IFFT,其所需总乘次数为：

$$\begin{aligned} m_c &= N + M + L + L \log_2 L \\ &= 2(N + M) - 1 + (N + M - 1) \log_2 (N + M - 1) \end{aligned}$$

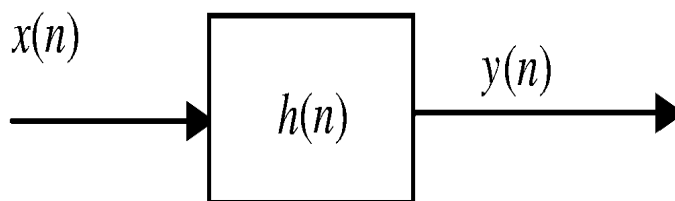
可以看出，当 N 和 M 较大时，FFT算法的Chirp-z变换速度会有很大的改进。



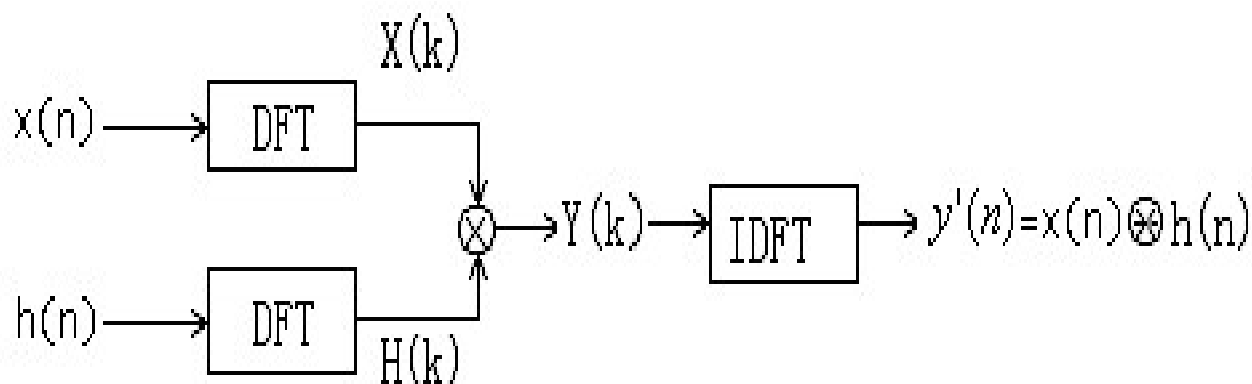
§ 4.7 线性卷积的FFT算法

FIR滤波器除了可以通过数字网络来实现外，也可以通过FFT变换来实现。

数字网络实现



FFT变换实现



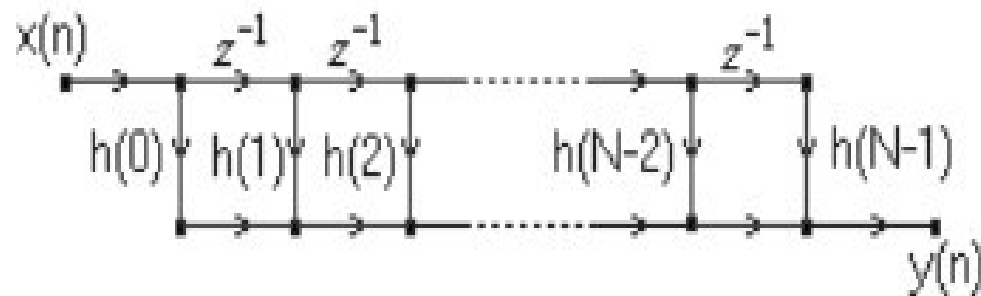


(a) 数字实现的滤波器

一个信号序列 $x(n)$ 通过FIR滤波器时，其输出 $y(n)$ 应该是 $x(n)$ 与 $h(n)$ 的卷积：

$$y(n) = x(n) * h(n) = \sum_{m=0}^{\infty} x(m)h(n-m)$$

在数字网络类的FIR滤波器中，普遍使用的横截型结构为：



这个横截型结构就是按照上面的卷积公式构造的，下面我们来统计一下这一卷积所需要的运算量。



设 $h(n)$ 的长度为 N_1 ，而信号 $x(n)$ 的长度为 N_2 ，即

$$h(n) = \begin{cases} h(n), & 0 \leq n \leq N_1 - 1 \\ 0, & n < 0, \quad n > N_1 - 1 \end{cases}$$
$$x(n) = \begin{cases} x(n), & 0 \leq n \leq N_2 - 1 \\ 0, & n < 0, \quad n > N_2 - 1 \end{cases}$$

卷积结果 $y(n)$ 也是一个有限长序列，长度为 $N = N_1 + N_2 - 1$

即

$$y(n) = \begin{cases} y(n), & 0 \leq n \leq N - 1 \\ 0, & n < 0, \quad n > N - 1 \end{cases}$$

根据FIR滤波器的横截型结构，可计算出线性卷积公式的运算量：

$$\text{乘法: } m_d = N_1 N_2 \quad \text{加法: } a_d = (N_1 - 1)(N_2 - 1)$$



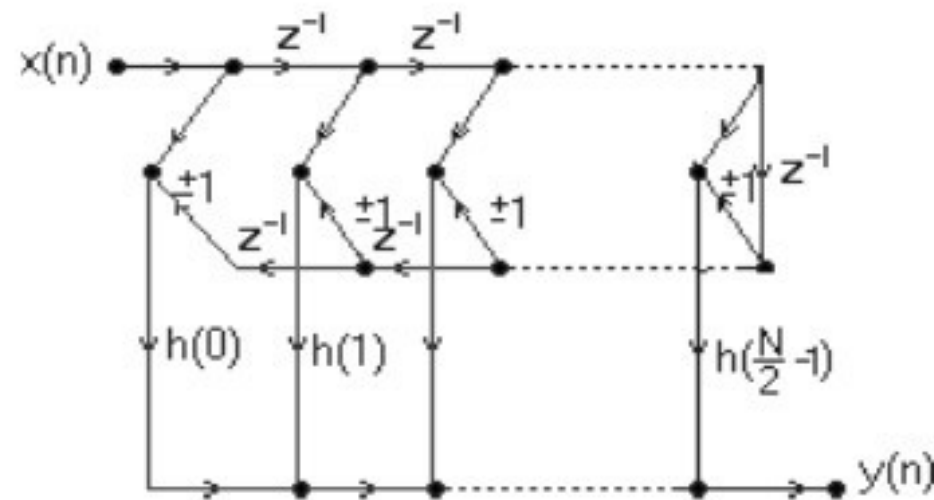
在FIR滤波器中，许多情况下都是采用线性相位滤波器的，线性相位滤波器由于其 $h(n)$ 具有偶对称或奇对称的特性，即

$$h(n) = \pm h(N_1 - 1 - n)$$

其横截型运算结构如下图所示，这里加权系数减少了约一半，所以对于线性想为FIR滤波器来说，相乘次数大约可减少一半，即

$$m_d = N_1 N_2 / 2$$

N_1 为偶数





(b) FFT实现的滤波器

第二章已经讲过，两个有限长序列的卷积可以用它们的圆周卷积来代替，圆周卷积结果不产生混淆的条件是使 $x(n)$ 、 $h(n)$ 都至少补零加长到 N 点，即 $N = N_1 + N_2 - 1$

$$h(n) = \begin{cases} h(n), & 0 \leq n \leq N_1 - 1 \\ 0, & n < 0, \quad n > N_1 - 1 \end{cases}$$

$$x(n) = \begin{cases} x(n), & 0 \leq n \leq N_2 - 1 \\ 0, & n < 0, \quad n > N_2 - 1 \end{cases}$$

这样 $y(n)$ 的值就可以通过以下四步来获得：

(1) 求 $H(k) = \text{FFT}[h(n)]$, N 点

(3) 运算 $Y(k) = H(k)X(k)$

(2) 求 $X(k) = \text{FFT}[x(n)]$, N 点

(4) 反变换 $y(n) = \text{IFFT}[Y(k)]$

从这四步可以看出，大部分工作量都可以用FFT完成。 77



假定 N 是以2为基数的整数幂，从以上四步看需要三次FFT运算，但对FIR滤波器来说，第一步实际上是不需要的，因此实际需要两个FFT运算，共需 $2\left(\frac{N}{2}\log_2 N\right)$ 次乘和 $2N(\log_2 N)$ 次加，另外第三步还需要 N 次相乘，因此总的运算量为：

$$m_F = N \log_2 N + N = N(\log_2 N + 1)$$

$$a_F = 2N \log_2 N$$

以线性相位FIR滤波器的相乘运算来比较两种方法的快慢。将两种方法相乘运算的工作量相比较，定义：

$$\begin{aligned} c_m &= m_d / m_F = N_1 N_2 / [2N(\log_2 N + 1)] \\ &= N_1 N_2 / \{2(N_1 + N_2 - 1)[1 + \log_2 (N_1 + N_2 - 1)]\} \end{aligned}$$



当 $x(n)$ 、 $h(n)$ 长度差不多时，例如 $N_1 = N_2, N = 2N_1 - 1 \approx 2N_1$,

此时, $c_m = N_1 / [4(\log_2 2N_1 + 1)] = N_1 / [4(\log_2 N_1 + 2)]$

如 $N_1 = 4, c_m = 8 / 4(2 + 3) = 1 / 2.5$

$N_1 = 32, c_m = 32 / 4(2 + 5) = 1.1$

$N_1 = 512, c_m = 512 / 4(2 + 9) = 11.6$

$N_1 = 4096, c_m = 4096 / 4(2 + 12) = 73$

由此可知，当 N_1 超过 32 后， N_1 越长，圆周卷积的优越性越大，因此常将圆周卷积称为快速卷积。

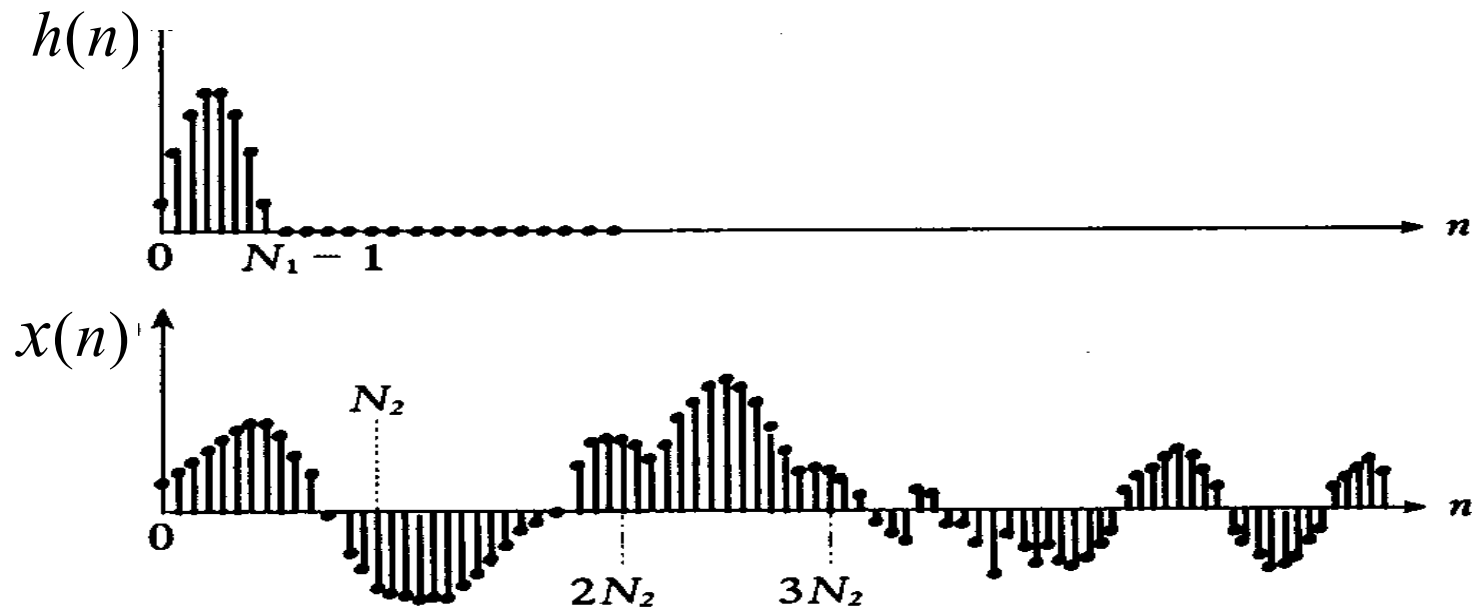
当信号 $x(n)$ 很长时，即 $N_2 \gg N_1$, $N \approx N_2$ ，此时

$$c_m = N_1 / [2(\log_2 N_2 + 1)]$$



显然 N_2 太长时, 会使 C_m 下降, 从而使圆周卷积的有点不能发挥出来克服这个困难的办法是采用分段卷积或称为分段过滤的办法。

当 $x(n)$ 是一个长序列时, 用周期卷积之所以不利是因为 $h(n)$ 必须补很多的零点, 以至于 N 点 $h(n)$ 中大部分是零, 很不经济。要改进它就必须将 $x(n)$ 分为一段段和 $h(n)$ 长度相仿的片段。如下图所示, 将 $x(n)$ 分为一段段长为 N_2 的片段, 然后对每一段 $x(n)$ 再分别用FFT来处理, 处理办法用两种: 重叠相加法和重叠保存法。





(a) 重叠相加法

假定以 $x_i(n)$ 表示上图中第 i 段 $x(n)$ 序列:

$$x_i(n) = \begin{cases} x(n) & iN_2 \leq n \leq (i+1)N_2 - 1 \\ 0 & \text{其它 } n \end{cases}$$

则输入序列为: $x(n) = \sum_{i=-\infty}^{\infty} x_i(n)$

这样输出也可分解为:

$$y(n) = x(n) * h(n) = \sum_{i=-\infty}^{\infty} x_i(n) * h(n)$$

其中: $y_i(n) = x_i(n) * h(n)$

上式表明, 只要将 $x(n)$ 的每一段分别与 $h(n)$ 卷积, 然后再将这些卷积结果相加就可得到输出序列。



这样，每一片段的卷积都可由快速卷积方法来计算，先对 $h(n)$ 及 $x_i(n)$ 补零，补到具有 N 点长度：

$$N = N_1 + N_2 - 1$$

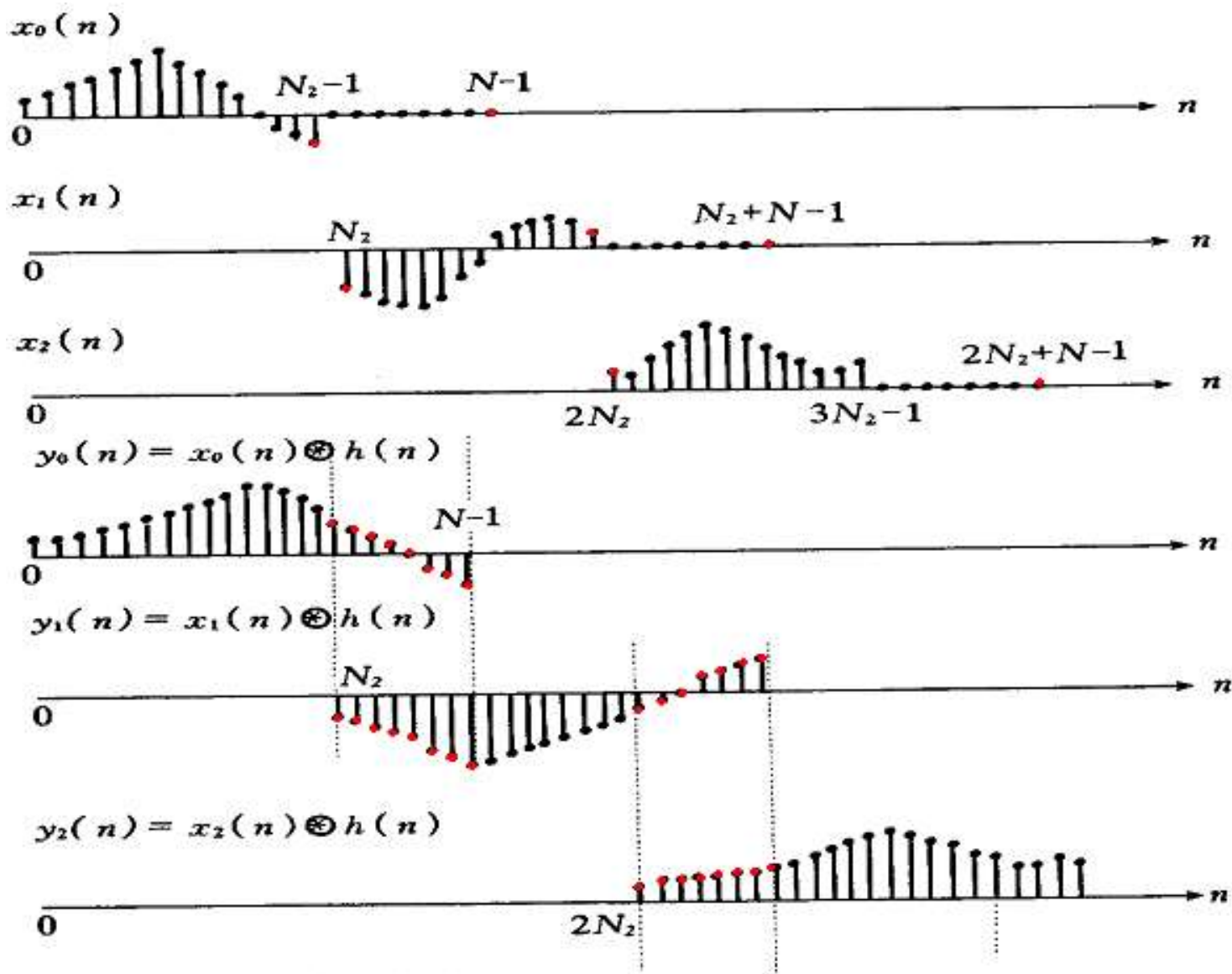
一般总是将 N 选为 $N = 2^M$ ，然后用以2为基数的FFT运算来求 $y_i(n)$ ：

$$y_i(n) = x_i(n) \otimes h(n)$$

由于 $y_i(n)$ 的长度为 N ，而 $h(n)$ 长度为 N_2 ，因此相邻两段 $y_i(n)$ 必有 $N - N_2 = N_1 - 1$ 的部分重叠。



重叠相加处理过程如下图所示：



重叠相加法



重叠相加法的整个运算分五步完成：

(1) 事先准备好滤波器的参数 $H(k) = \text{DFT}[h(n)]$, N 点

(2) 用 N 点 FFT 运算 $X_i(k) = \text{DFT}[x_i(n)]$

(3) $Y_i(k) = X_i(k)H(k)$

(4) 用 N 点 IFFT 运算 $y_i(n) = \text{IDFT}[Y_i(k)]$

(5) 将重叠部分相加起来

$$y(n) = \sum_{i=-\infty}^{\infty} y_i(n)$$



(b) 重叠保存法

如果将上面分段序列中的补零部分不补零而是保存原来的输入序列值，那么显然圆周卷积以后将出现混淆现象。

这个混淆部分只发生在 $y_i(n)$ 的起始一段：

$$\begin{aligned} y_i(n) &= x_i(n) \otimes h(n) \\ &= \sum_{m=0}^{N-1} x_i(m) h((n-m))_N R_N(n) \end{aligned}$$

由于 $h(n)$ 的长度为 N_1 ，因此当 $0 \leq n \leq N_1 - 2$ 时，将在 $h((n-m))_N$ 的尾部出现非零值。



先将 $x(n)$ 分解为

$$x_i(n) = \begin{cases} x(n + iN_2 - N_1 + 1), & 0 \leq n \leq N-1 \\ 0 & \text{其它 } n \end{cases}$$

然后利用FFT算法求出 $y_i(n) = x_i(n) \otimes h(n)$,

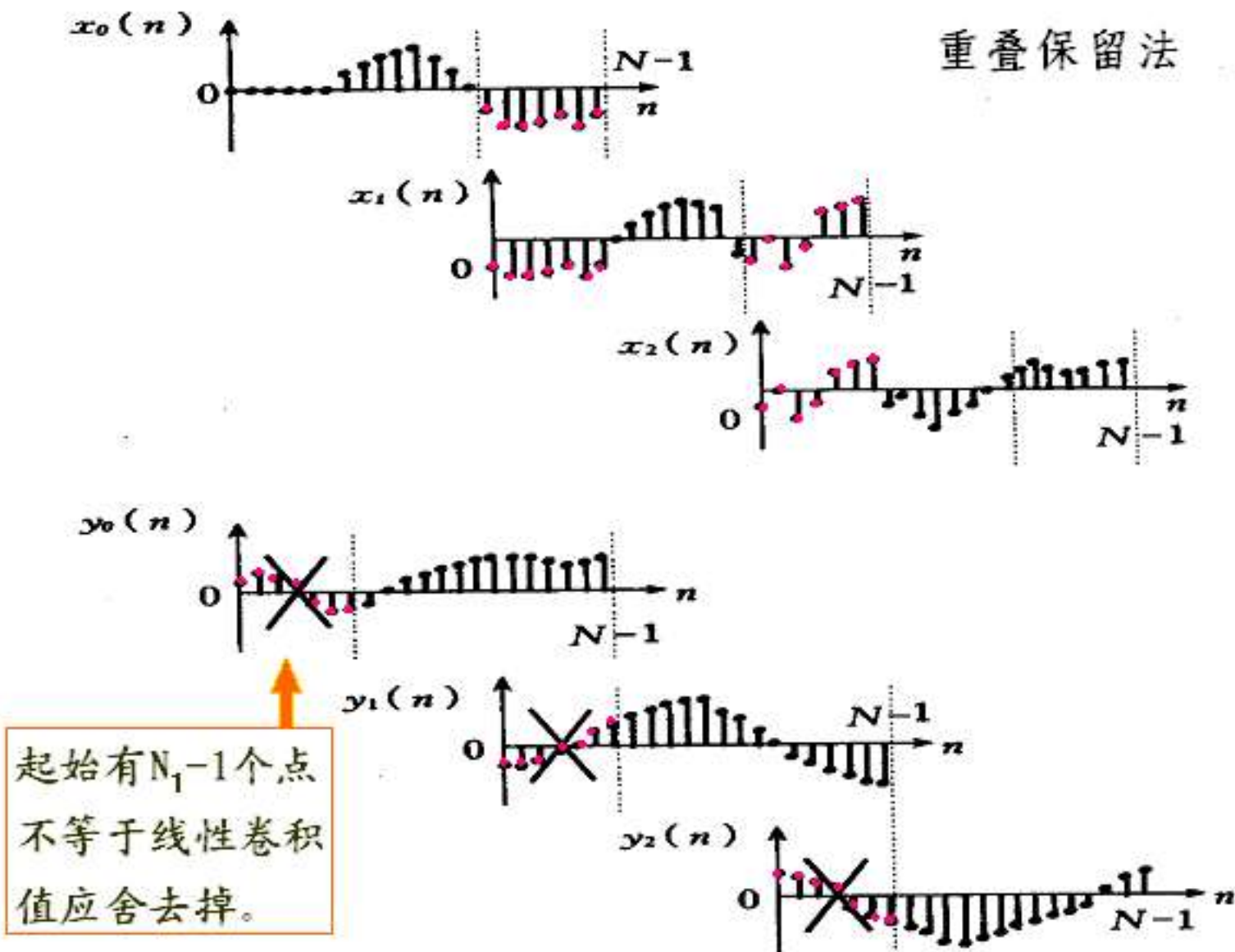
再抛弃 $y_i(n)$ 的前 $N_1 - 1$ 点, 最后将各 $y_i(n)$ 顺次连接起来就得到输出序列 $y(n)$ 。



重叠保存法和重叠相加法工作量差不多，但可以省去重叠相加法的最后一道相加运算。一般用FFT过滤信号只用于FIR滤波器阶数大于32的情况，且 N_2 一般选择 N_1 的5到10倍左右，即 $N_2 \approx (5 \sim 10) N_1$ ，这样可接近最高效的运算。



重叠保存处理过程如下图所示：





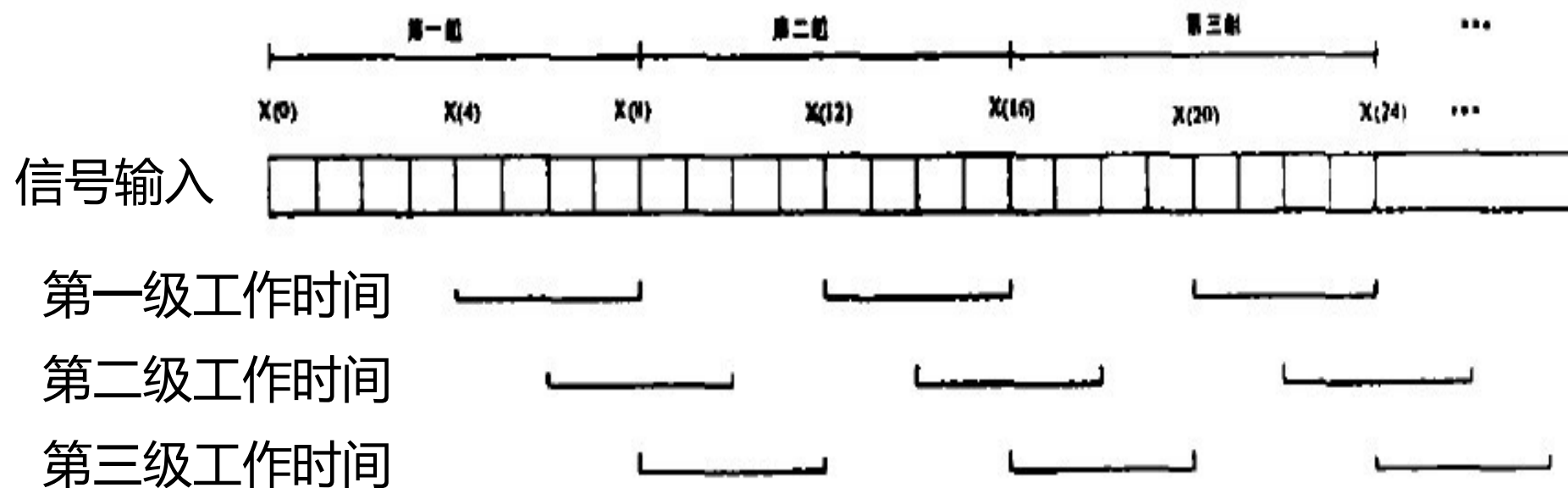
§ 4.8 FFT的流水线工作原理

在信号实时处理中，要求信号输入的同时，及时地、逐段地完成对信号的FFT运算。完成一个N点的FFT运算需 $m_F = \frac{N}{2}(\log_2 N)$ 次复乘，因此实时工作时，计算机的速度至少必须高于每采样间隔执行 $m_F / N = \frac{1}{2}(\log_2 N)$ 次复乘。当信号速率很高时，数字硬件往往难以达到那样高的速度，解决办法之一就是在设计专用FFT处理器时采用流水线工作方式。

FFT算法的一个重要特点好似具有分级运算结构，因此，为提高运算速度可以在每级运算中采用单独的运算器，第一级运算器算完后送给第二级，自己再算新来的数据，第二级算完后送第三级……，如此形成流水线的工作方式。



下面以 $N=8$ 的FFT算法为例介绍流水线工作原理：



由上图可知，完成 $N=8$ 点FFT运算需要三级流水线，信号输入时，开始必须将前四个数据寄存在第一级的存储器中，因为第一级运算器只有在 $x(4)$ 到来之后才能看是运算第一个蝶形结， $[x(0)、x(4)]$ 算完后将数据送入第二级存储器，如果



计算器的速度能满足每采样周期完成一个蝶形结运算（即1次乘法/采样周期），则当 $x(5)$ 到来时，就可以及时地计算 $x(5)$ 对应的蝶形结

$[x(1)、x(5)组]$ ，...，这样，在第二组8点到来以前，第一级运算器正好可以完成它对第一组信号的全部运算，第二组信号再像第一组信号一样，开始先寄存钱一半数据，等 $x(12)$ 到达时，第一级运算器又开始下一个循环的运算。

同样，第二级、第三级的工作由于蝶形结配对的要求，都必须等前一级工作一半以后才能开始工作，如果各级都按同一节拍工作，则每一级都可以在半个周期的时间内完成各自的运算，因而整个运算可以连续进行下去。一个 N 点FFT流水线运算，需要 $M = \log_2 N$ 级运算器，每级的速度只要求1次乘法/采样周期就够了，从而大大降低了对数字硬件的速度要求，或者同样的硬件可以完成更高速度的信号处理。这样的代价是运算器需要增加 M 倍，存储单元也要增加约 $M/2$ 倍，另外，最后所获得的结果也必须有一定的时延。



从FFT流水线工作示意图中可以看到，每级运算器实际上只有一半时间在工作，工作效率只有一半，这是因为一个 N 点FFT要求运算速度为 $\frac{1}{2} \log_2 N$ 次乘/采样周期，这样的运算量被速度为1次乘/采样周期的运算器分担时，只要 $\frac{1}{2} \log_2 N = \frac{M}{2}$ 个就够了，或者说有 $\log_2 N$ 个运算器分担的话，每个运算器的速度只要目前的一半就够了，这说明即使采用流水线方案也还有一般的潜力可以挖掘，这种潜力往往可以根据处理信号的具体要求加以进一步利用。



第四章作业

4.18; 4.19